2022 18th European Dependable Computing Conference (EDCC) | 978-1-6654-7402-3/22/$31.00 ©2022 IEEE | DOI: 10.1109/EDCC57035.2022.00025

RPUGuard: Real-Time Processing Unit Virtualization for Mixed-Criticality Applications

Marcello Cinque, Gianmaria De Tommasi, Sara Dubbioso, Daniele Ottaviano
Dept. of Electrical Engineering and Information Technology
University of Naples Federico II
Naples, Italy
{macinque, detommas, sara.dubbioso, daniele.ottaviano}@unina.it

***Abstract*—Virtualization techniques are attracting interest in industrial applications to implement mixed-criticality systems taking full advance of modern embedded architectures, such as Multi-Processor Systems on Chip (MPSoCs). However, the virtualization support for important hardware accelerators, such as Real-Time Processing Units (RPUs), used for real-time and/or safety-critical workloads, is still overlooked. In this paper, we propose an architectural solution to virtualize RPUs in complex embedded systems. Specifically, we design and implement a component, named RPUGuard, which is able to guarantee isolated communication channels with a fixed bandwidth between virtual machines, running on regular Application Processing Units (APUs), and the RPU on the same MPSoC. We evaluated RPUGuard on the Zynq Ultrascale+ board, in the context of a challenging case study concerning the magnetic control system of the ITER experimental nuclear fusion reactor. Results demonstrate how our solution can mitigate the weaknesses of current asymmetric communication techniques, while providing isolation guarantees to critical communication channels.**

***Index Terms*—Mixed-Criticality, Real-time Processing Unit, Embedded Virtualization, Real-Time systems, MPSoC, Fusion engineering**

I. Introduction

The continuous growth of interest by industry and academia on the Industrial Internet of Things (IIoT) has paved the way for the development of new technologies aiming to meet the stringent functional and non-functional requirements of the IIoT [1] [2]. For instance, Fog Computing [3] is emerging as a solution in the IIoT to move computing services closer to the industrial equipment, to so called *edge devices* (or edge nodes), in order to meet performance, timeliness, and predictability requirements. In this scenario, edge devices are expected to grant several non-functional properties, such as scalability, agility, reconfigurability and workload consolidation, for efficient deployment, other than dependability, security and *mixed-criticality*, that is, systems capable of hosting on the same hardware both critical real-time applications and value-added non-critical services [4].

To meet such stringent requirements, large hardware manufacturers such as Intel and AMD (Xilinx) started to produce integrated systems characterized by heterogeneous components: the MPSoC [5]. Many industries have then decided to use MPSoCs as edge nodes to reduce space, weight, power, and cost (SWaP-C), as witnessed by [6] [7] [8] [9]. These SoCs are characterized by heterogeneous hardware components such as APUs for general-purpose computation, FPGAs (Field Programmable Gate Array) for high-performance computing (HPC), RPUs for real-time and safety computation, and GPUs (Graphic Processing Units) for parallel and graphical computations. The heterogeneity of these SoCs guarantees high performance, scalability, and reconfigurability, but their inherent complexity makes them not easy to use.

One of the most promising technologies considered today to streamline the adoption of MPSoCs is *virtualization* [1] [10] [11] [12] [13]. The idea is to bring the benefits of virtualization and Virtual Machines (VMs), largely adopted for cloud services, to embedded edge nodes, so as to guarantee:

- *Hardware Consolidation*: multiple VMs on a single board ensure optimal utilization of hardware resources.
- *Legacy Software Migration*: legacy software can be migrated as a VM on a board without any modification saving development cost and time to market.
- *Improvement of Manageability*: VMs can be tested on different boards and then migrated to the target board ensuring easy deployment. Moreover, each VM can be launched and stopped at run-time providing high scalability. VMs can also be orchestrated and replicated on the same board to improve availability and reliability.
- *Mixed-Criticality*: virtualization allows embedded real-time software, e.g.,a real-time operating system (RTOS), to run in parallel with modern commodity OSes that can offer value-added services to safety-critical real-time software [14] [15] [16].

The transition of cloud virtualization techniques to the embedded world is not an easy task, due to the high heterogeneity of hardware and software technologies used in the

This work has been carried out within the framework of the EUROfusion Consortium, funded by the European Union via the Euratom Research and Training Programme (Grant Agreement No 101052200 - EUROfusion). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them. This work was also partially supported by Italian MIUR under PRIN grant #20177BZMAH, by the project COSMIC of UNINA DIETI and by the R&D project "REINForce: REsearch to INspire the Future" (CDS000609), funded by the Italian Ministry for Economic Development (MISE).

978-1-6654-7402-3/22/$31.00 ©2022 IEEE
DOI 10.1109/EDCC57035.2022.00025

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

embedded scenario [17] [18]. Hypervisors (i.e., the software components abstracting the hardware to VMs) must therefore be extended to take into account all architectural resources, including heterogeneous computational units and accelerators such as FPGA, GPU, and RPUs. Although FPGA and GPU virtualization does not yet have a universally adopted standard solution, they are starting to be addressed in the literature, as we discuss in Section II.

On the other hand, to the best of our knowledge, no solutions to date are available regarding the sharing of RPUs among multiple running VMs. Thus, if two VMs are willing to (simultaneously) offload real-time processing to the RPUs, the resulting behavior can be unpredictable, without the proper coordination and abstraction provided by the hypervisor. The absence of any kind of management in this case can be particularly dangerous, as RPUs are used to execute real-time safety-critical code whose failure could cause considerable damage.

To realize a dependable solution for RPU's virtualization, the following questions need to be answered:

- *[RQ1] Multi-Tenancy Abstraction*: How can an hypervisor abstract the RPUs to more than one VM, making the latter believe to be the only one using it?
- *[RQ2] Communication Technologies*: Are the current Asymmetric Multi-Processing (AMP) communication technologies (adopted for inter-processor communication in MPSoCs) appropriate to realize deterministic and reliable applications with strict temporal constraints?
- *[RQ3] Isolation Methodologies*: How can an hypervisor manage parallel communication channels ensuring isolation and assure a certain amount of communication bandwidth to each channel (useful for timing analysis and incremental development and certification)?

Aiming to answer to above questions, the main contributions of this work are: (1) a novel architectural solution for RPU sharing in mixed-criticality applications, trough virtualization; (2) the development and experimentation of a bandwidth regulation method to provide isolation in multichannel transmissions between APUs and the RPU.

Our experiments on real hardware show the shortcomings of currently available AMP technologies and the benefits of the proposed solution in the context of a compelling case study related to the Vertical Stabilization (VS) of plasma in the ITER experimental fusion power reactor [19].

This paper extends our previous work on RPU virtualization [20] providing a complete and organic architecture design, clear answers to research questions coming from an in-depth analysis of RPU virtualization problems, and also more details about the plasma magnetic control system case of study, of which the VS is just one of the components. Finally, the article presents novel experimental results on AMP communication issues.

The rest of the paper is organized as follows. In Section II we review the most modern accelerator virtualization techniques, with emphasis on GPU and FPGA virtualization. Section III describes the architectural solution design, whereas Section IV presents the ITER case study. Furthermore, in Section V we show several practical results to answer research questions about the reliability and isolation of AMP communication. Conclusive remarks and future works are provided in Section VII and VI respectively.

II. Related Work

Recently, there has been a great deal of commitment to developing new hypervisors for embedded systems and for real-time purposes [21] [22] [23] [24] [25] [26]. These solutions aim to take advantage of virtualization to solve SWaP-C related problems. The common idea is to implement on a single board many services with different levels of criticality decreasing the number of total boards needed (SWaP-C) without loss in performance and isolation. Despite all this effort and the heterogeneity of new approaches and solutions, accelerator support is still immature, as shown in [1].

A. Accelerators Virtualization

There are many techniques to virtualize the GPU and the FPGA in server environment, such as fixed pass-through, device emulation, or hardware support virtualization, but in all these cases the solutions do not focus on isolation and minimal impact, but rather on performance. Therefore, porting it to an embedded system with real-time requirements requires effort.

Virtualizing GPUs is a relatively new area of study, and although there are several proposed solutions [27], it remains an open challenge. This is not only due to the heterogeneity of the GPU's architectures but also because GPU drivers are not open for modification due to intellectual property protection. All these motivations make conventional virtualization techniques not directly applicable for GPU virtualization.

Virtualization of FPGAs is also a very complex area of research [28] [29]. It is possible to distinguish two approaches to FPGA virtualization in the literature: the FPGA used to accelerate hypervisor operations [30] [31], and the FPGA used as a shared hardware accelerator resource between VMs [32].

Both GPU and FPGA virtualization techniques are not suitable for virtualizing the RPU since both are very hardware specific. Furthermore, while GPU and FPGA are used to accelerate calculations to achieve higher performance, the RPU is used to achieve high determinism. It is therefore essential to develop virtualization techniques that focus on the requirements of predictability and isolation taking into account the strengths and limitations of the RPU architecture.

III. Envisioned Solution

A. Architectural Design

The envisioned system model foresees several VMs with different levels of criticality that require RPU services. The requested services can be very heterogeneous since the RPU, unlike GPU and FPGA, is a fully functional processor unit. The RPU does not simply accelerate calculations, but it can implement more or less complex logic to manage incoming messages. The advantage is that the code executed on this kind of processor is expected to be more easily predictable and

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

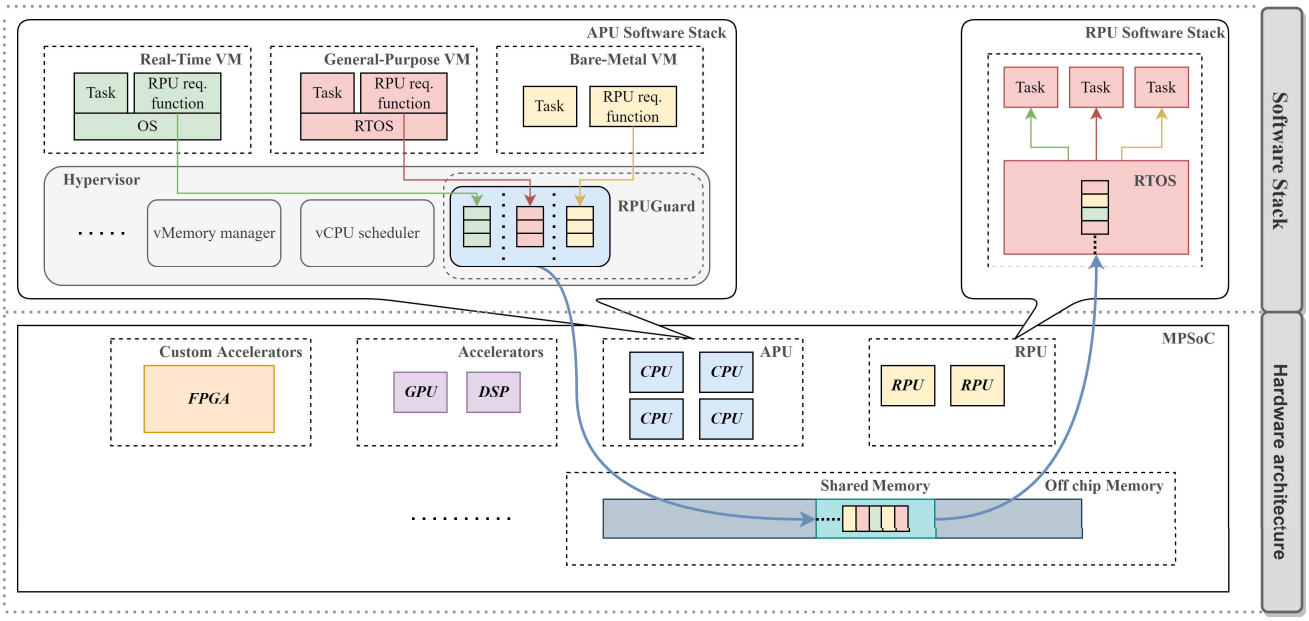

Fig. 1. Architectural Design

more suitable for performing safety-critical tasks as the hardware is certified (e.g., the ARM Cortex-R5 RPU can be used in safety-critical applications up to SIL 3 according to IEC 61508 and up to ASIL D according to ISO 26262). Hence, before describing the architectural design of the solution, we must define some assumptions.

The first assumption (*A1*) states that each VM is aware of the code running on the RPU. This assumption is reasonable since typically, in embedded systems, applications that will be performed are well known before the execution, especially in a safety-critical environment with strict temporal requirements.

The second assumption (*A2*) states that the software stack executed on RPU is realized taking into account that more than one VM may require its services. Therefore, the RPU's software stack must manage its load guaranteeing the worst-case execution time of all exposed services.

Fig. 1 shows the high-level design architecture for RPU virtualization. Starting from the APU software stack, we imagine a number of VMs running different OSes or Bare-Metal applications characterized by different levels of criticality. However, each VM has at least one function that requires RPU usage, using an inter-processor communication mechanism. Even if assumptions A1 and A2 are respected, concurrent requests by VMs can still introduce non-predictable effects and interferences on critical tasks, as will be confirmed by our experiments in response to *[RQ2]*. Therefore, we need to introduce a component in the hypervisor that manages requests: The *RPUGuard*. It uses a shared memory to exchange messages with the RPU software stack, as typically done by AMP communication mechanisms. To answer to research questions *[RQ1]* and *[RQ3]*, the RPUGuard mechanism must capture all messages from and to the RPU, establishing an isolated communication channel for each VM.

B. RPUGuard

Algorithm 1 RPUGuard: feasibility_check

$BW_{tot} \leftarrow$ Maximum bandwidth
$BW_{new} \leftarrow$ New VM bandwidth request
if $BW_{tot} \geq (\sum_{i=0}^{n-1} BW_{vm}(i) + BW_{new})$ **then**
    Create new communication channel
    $n \leftarrow n + 1$
    $BW_{extra} \leftarrow (BW_{tot} - \sum_{i=0}^{n-1} BW_{vm}(i))$
    The new VM can access the RPU
else
    The new VM cannot access the RPU
end if

Given the maximum communication bandwidth BW_{tot} of the channel, RPUGuard is in charge of partitioning it, ensuring a certain amount to each VM. When a VM starts, it can specify the maximum bandwidth BW_{vm} needed to communicate with the RPU during its execution, and RPUGuard can accept or refuse the request; granting or denying access to the RPU from the beginning of the execution.All VMs that start without any bandwidth request are managed with a best-effort policy by RPUGuard; to these VMs is reserved a bandwidth equal to $BW_{extra} = (BW_{tot} - \sum_{i=0}^{n-1} BW_{vm}(i))$ where n is the number of VMs which did require bandwidth guarantees at the start time. Then, only if a new VM starts specifying a bandwidth, RPUGuard must perform a feasibility check (see Algorithm 1) to verify that the new VM can be added without

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

reducing the communication bandwidth of running VMs below their designed limits. If the feasibility check is completed successfully, RPUGuard manages the bandwidth assigned to each VM to include the new communication channel. When a VM mistakenly uses more bandwidth than it was suppose to (BW_{VM}), the RPUGuard must handle the requests. Hence, every time a new send message request to RPU arrives, the RPUGuard checks the elapsed time (T_{el}) since the last request from the same VM. If the elapsed time is below the allowed time interval (T_{min}) assigned to the VM, the request is delayed for a time interval equal to the difference $T_{min} - T_{el}$ plus an other time interval T_{min} for each request not yet served. Doing so, RPUGuard prevents the VMs to send messages with a frequency higher than those set at start time (see Algorithm 2).

Algorithm 2 RPUGuard: send_message

1: $R_n \leftarrow$Number of requests in queue
2: $T_{el} \leftarrow$ elapsed time since the last send message
3: $T_{min} \leftarrow$ minimum interval time between requests
4: $T_d \leftarrow (max(0, T_{min} - T_{el}) + (R_n - 1) * T_{min})$
5: delay the request for a time T_d
6: Send the message to RPU

C. Implementation Details

The proposed architecture design requires a message-based communication technique that makes use of shared memory. Recently, practitioners of AMP systems rely on an open-source framework named *OpenAMP* [33] to realize communication between asymmetric processors. OpenAMP is built on top of two other frameworks: *RemoteProc* and *RPMsg* (Remote Processor Messaging). RemoteProc is used to manage the lifecycle of a remote processor.; On the other hand, RPMsg is a messaging mechanism implemented on top of the *VirtIO* framework [34] to realize inter-processor communication. OpenAMP is currently the most used technology to exchange messages with the RPU because it is highly portable and open source, but this framework alone is still not enough to ensure isolation and transparency to VMs running on APU cores. Our implementation makes use of the OpenAMP framework, but we implement the RPUGuard mechanism to guarantee isolation and transparency to VMs. Since our solution uses OpenAMP and more specifically RPMsg, we provide some further details on RPMsg in the following.

The RPMsg framework implements AMP transmission using three independent layers: *Transport*, *Media Access Control*, and *Physical Layer*. The *Physical Layer* is the most straightforward and comprises only two hardware components: shared memory and interprocessor interrupts (IPI). The shared memory is accessible by both the cores and each time a new message is sent, an IPI is called to warn the remote core. The central layer, or *Media Access Control* layer, uses the *VirtIO* queue mechanism: each virtqueue contains two ring buffers named "Available" and "Used". The Master core allocates the "used" ring buffer, writes inside them, and adds them to the "available" ring buffer. The remote core reads the data from

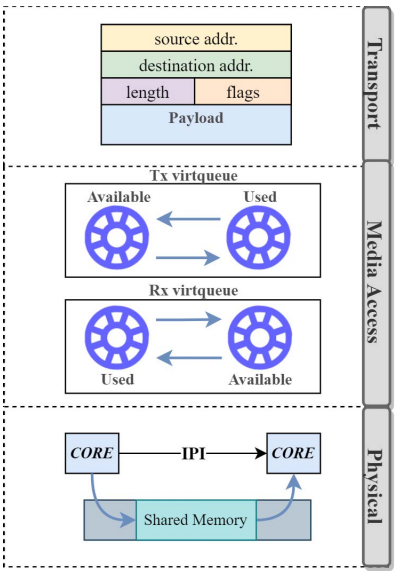

Fig. 2. RPMsg layers

the "available" ring buffer, processes them, and finally returns them to the "used" ring buffer. To implement the double-side communication two virtqueues must coexist in each channel, one to send messages (Tx virtqueue) and one to receive them (Rx virtqueue). Finally, the *Transport* layer simply defines a message format as shown in Fig. 2.

Usually, each remote core in the RPMsg component is represented by a RPMsg *device*, also known as RPMsg *channel*. To implement the envisioned multichannel communication, we could use an entire RPMsg virtual device, with its two virtqueues, for each VM. However, this solution has two intrinsic limitations: The RPU firmware, before the system starts, has to maintain the memory area needed for each RPMsg device. So, (I) the number of channels that VMs will require must be known in advance, limiting the flexibility of virtualization, and (II) it causes a waste of memory that is not desirable in an MPSoC with limited hardware resources.

An alternative solution consists of using RPMsg endpoints that are logical connections on top of the RPMSg channel. Endpoints allow the user to bind multiple Rx callbacks on the same channel distinguished by the destination address of the messages.Our solution uses endpoints, but adjusts the order and frequency of messages sent between VMs via the RPUGuard component implemented in the hypervisor.

Before we dive into the experimental results, the next section presents an in-depth analysis of our case study about the VS problem, to evaluate our approach in complex scenarios such as plasma magnetic control in fusion reactors.

IV. The Case of the ITER Magnetic Control Systems

Nuclear fusion is foreseen as a possible source of energy for the next century [35]. To obtain the fusion reaction on Earth, a fully ionized gas, called *plasma*, must be confined and

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

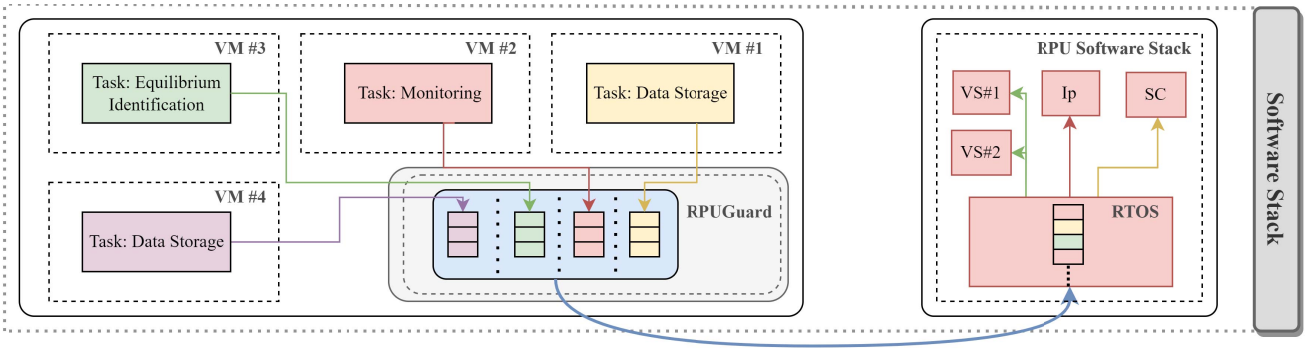

Fig. 3. Software Architectural Design for the ITER VS case

heated to temperatures up to hundreds of millions of degrees. Under these conditions, the Coulomb repulsive force can be overcome, and collisions between ions can result in fusion reactions [36].

The most promising experimental devices to prove the feasibility of nuclear fusion are tokamaks: designed with a toroidal shape, they are able to magnetically confine a plasma ring. The ITER tokamak, which is under construction in France, is the most ambitious of many International projects that have been successfully established to operate tokamak machines around the world. It is foreseen to be the first fusion device to produce net energy.

The complexity of ITER is an example of a large scale complex system where thousands of applications have to be integrated on a limited number of interconnected machines, with a real-time infrastructure that calls for the integration of several control loops, with different sampling times and different requirements in terms of reliability [37]. The requirements to be taken into account during the design of the real-time infrastructure of such large experimental plant have been discussed in [18].

Modern tokamaks to be operated require the implementation of several challenging control schemes. The *magnetic control system* is in charge of controlling the current induced into the plasma, as well as its shape and position, by regulating the magnetic field generated by the currents that flow in several external coils. Moreover, the elongated shape of high performance plasma pursued in modern tokamak unfortunately turns them to be vertically unstable. Therefore an active VS system has to be included in the magnetic control architecture (the interested readers may refer to [38] for more details about magnetic control problems and functional architecture).

Therefore magnetic control is an essential and critical component, that is needed since day one to operate a tokamak. Hence, the ITER magnetic control is a valid example of an advanced control system that may require a mixed-criticality system (MCS) solution. Both implementation and commissioning of various magnetic control aglorithms can benefit from a MCS solution, encompassing several VMs and RPU workloads on the same board.

In Fig. 3 is reported a possible implementation of the RPU virtualization mechanism proposed in this paper applied to the ITER magnetic control system. Since we are considering the point of view of the controller applications, where there is no concern about the physical hardware on which they will be effectively run, Fig. 3 shows only a possible software architecture. Going into details, some distinctive use cases in the context of the plasma magnetic control can be inferred:

- UC#1: open-loop commissioning of the software implementation of a control algorithms. During a tokamak pulse new versions of the various control algorithms can be tested on the plant without closing the loop on the actuators, while the *standard* (and already commissioned) algorithms are actually controlling the plasma column, being the ones running in closed-loop. Therefore, it is possible to find several implementations of a given control algorithm in the RPU software stack (Fig. 3 shows the case of two VS algorithms running in parallel). The open-loop validation is a standard commissioning procedure for control algorithms, which aims at verifying the input-output behaviour of the software implementation.
- UC#2: magnetic control system components running in isolation. The control algorithms needed for the plasma magnetic control reflect their different purposes in the different sampling times and reliabilities required. Such that, the RPU can host all the magnetic control components, i.e. the plasma current (I_p) controller, the shape controller (SC) and the VS, assuring execution capabilities and isolation of faults for each system.
- UC#3: non real-time support functionalities implemented on APU on the same MPSoC. Every tokamak pulse is a complex scientific experiment; its execution has to be monitored and every variable and command stored to be analysed afterward. Therefore, support functionalities are needed, i.e., logging and data storage. These can be implemented on APUs in communication with real-time software.
- UC#4: execution of computationally demanding tasks. Most of the plasma parameters of interested for control cannot be directly measured, therefore computationally

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

demanding (and usually iterative) reconstruction codes are needed to estimate such parameters. One example is the reconstruction of the so called plasma *growth rate* γ to deploy adaptive VS systems. Indeed, during a tokamak discharge the plasma configuration changes, and so does also the linearized model that describes the plasma behaviour. Since effective VS solutions call for adaptive mechanism based on the estimation of γ which is calculated by using the plasma linearized model, equilibrium reconstruction codes and the corresponding linearization procedure [39] need to be executed during the tokamak discharge. Given the typical computational burden needed by such a task, this should be executed by the APU (possibly with the support of GPUs), without interfering with real-time control. Moreover, modern magnetic control algorithms may requires the deplyment of AI-based control policies ([40], [41]) that call for the usage of resources other then just the RPUs.

V. Experimental Results

Table I shows the details of our experimental platform. We decided to use the Zynq Ultrascale+ platform not only because it is a widely used MPSoC for IIoT applications, but also because it has been chosen as the target board for the implementation of several diagnostic and control systems at ITER. On the APU, we decided to deploy the official Linux kernel provided by Xilinx (linux-xlnx) patched with the PREEMPT-RT patch, in order to reduce as much as possible the latency introduced by the operating system. On RPU, we run freeRTOS which is a widely used open-source real-time operating system, and is compatible with OpenAMP.

TABLE I
Experimental Platform.

Name	Description
Platform	Zynq Ultrascale+ 104
APU	Cortex-A53 (quad-core)
RPU	Cortex-R5 (dual-core)
APU O.S.	Linux-xlnx (Preempt-RT patch)
RPU O.S.	FreeRTOS

In Section V-A we show the bandwidth limitation of OpenAMP as a starting point for our experiments. In Section V-B we measure the message latency due to interference from parallel communication, pointing out the problems caused by exceeding the band boundary. Finally, in Section V-C the benefits of our solution are shown and the results are discussed.

A. OpenAMP Bandwidth limitation

Before testing the isolation of the communication channels, we experimentally tried to figure out the maximum bandwidth that OpenAMP can achieve using a single channel and a single endpoint, and which is its behaviour under stress. Given a fixed amount of memory dedicated to OpenAMP's virtqueues (262144 bytes, that is, the default OpenAMP virtqueues' size), we verify that if the frequency of message requests grows beyond a certain maximum limit, the queues fill up and messages begin to show high latency. The objective is to measure the maximum frequency of message transmission above which high latency occurs, for a given virtqueue size. To find this frequency, we implemented a periodic task on Linux that sends messages to the RPU using OpenAMP. The RPMsg message size is fixed by design to 512 bytes, where 496 bytes are used for the payload and 16 bytes are used for the header. Therefore, to measure the maximum bandwidth, each sent message fills the payload to its maximum in all experiments. Therefore, the transmission bandwidth is calculated by dividing 512, that is the number of sent bytes, by the send frequency in microseconds. As an example, if the frequency of activation of the task is $20us$ then the used bandwidth is $BW = \frac{512}{20} = 25,6\,MB/s$.

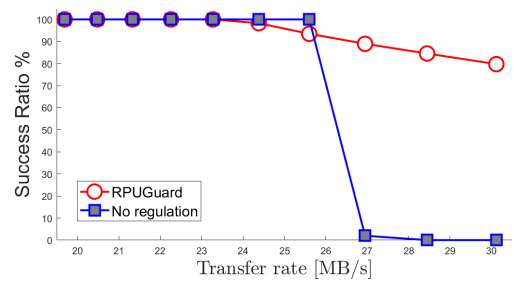

Fig. 4. Transfers success ratio

For our experiments, we consider the success ratios as the number of messages that are delivered in expected time within a time interval equal to two seconds. As we can see in Fig. 4, when the transfer rate is greater than $25,6MB/s$ OpenAMP, without any regulation, presents an impressive loss of performance; the success ratios decrease below 5%. This results answers *[RQ2]* showing the limits of current AMP communication technologies, which can not be used without regulation in safety critical real-time applications, since critical messages, sent by critical VMs, could not be delivered on time due to the requests of other concurrent VMs. However, with the RPUguard mechanism, despite the fact that the performance decrease starts at a transfer rate of $24.3MB/s$ due to the overhead introduced by the guarding mechanism, the success ratio decreases as expected considering the maximum bandwidth admitted by RPUGuard imposed at $25,6MB/s$. Therefore, to assure 100% of success rate, in the following experiments RPUGuard is set with a $BW_{tot} = 23,28\,MB/s$ and this bandwidth is then divided into several isolated communication channels with different requirements. Let us note that a different virtqueue size would impact the bandwidth measurement, so the desired bandwidth (up to reasonable memory limits) can be tuned by configuring the virtqueue size.

B. Isolation tests without regulation

In order to measure the data transfer latency caused by parallel communications, we deploy the ITER VS algorithm (critical task) together with a disturbance algorithm on our platform (control algorithm details can be found in [42]).

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

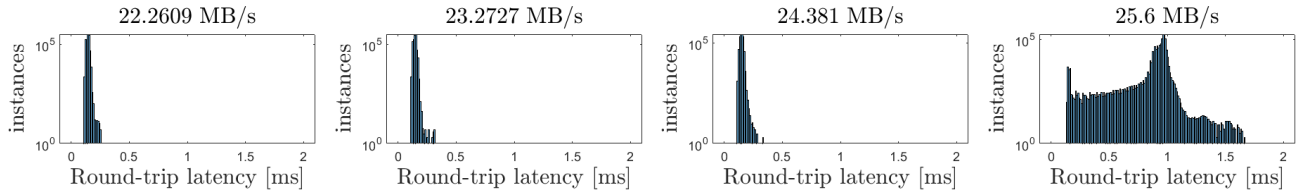

Fig. 5. Histograms of APU-RPU round-trip latency without regulation, as a function of the disturbance bandwidth

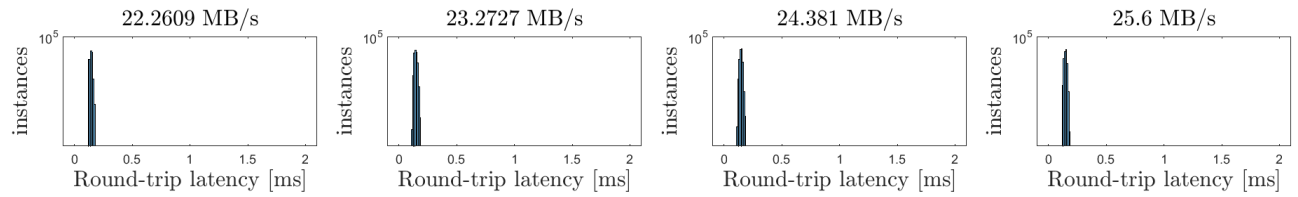

Fig. 6. Histograms of APU-RPU round-trip latency with RPUGuard, as a function of the disturbance bandwidth

The VS algorithm is implemented in the RPU and each millisecond receives from the APU the value of the plasma's current and velocity; it executes the control algorithm in a predictable way and sends back the voltage value to the APU as output. At the same time, a disturbance task deployed on the APU sends purposeless messages to a different service running on the RPU. So, in principle, the VS algorithm and its APU counterpart are completely separated from the disturbance tasks and related RPU service, so they should not interfere each other. The scenario is close to the *UC#4* discussed in section IV where the equilibrium reconstruction code is considered as a disturbance task.

The frequency of the disturbance task is incremented step by step, so we can verify how much this factor influences the task that carries out the VS algorithm. Therefore, we calculate the time between the transmission of the current and velocity values from the APU to the RPU and the moment when the voltage is returned.Considering that the execution time of the algorithm deployed on the RPU has a negligible variance, the differences that we can observe in the round-trip time are only due to the communication channel.

As expected, looking at the histograms of the latencies in Fig. 5, we can notice that, above a certain transmission bandwidth used by the disturbance task, the occurrences with high latency increase and some of them exceed the VS deadline (one millisecond).This happens when the disturbance algorithm uses a communication bandwidth of around $25.6\,MB/s$. This behaviour was expected since the communication bandwidth used by the VS algorithm is around $1.02\,MB/s$.The total bandwidth for our platform without any regulation is at $BW_{tot} = 25,6MB/s$ as previously stated, so, since the disturbance task uses all the available transfer bandwidth, the critical task sees its response time increase dramatically. This behavior is not acceptable in mixed-criticality systems, as the correct functioning of the VS critical task is not guaranteed.

C. Isolation tests with regulation

The same experiment seen in Section V-B is now carried out using the RPUGuard mechanism. Looking at Fig. 6, it is clear that the interference caused by the disturbance task is now perfectly mitigated. To ensure the independence of the communication channel at the software layer, we have assigned a maximum bandwidth of $23,28 - 1.02 = 22,26\,MB/s$ to the disturbance task, which corresponds to at most one request per $23\,us$, and we have assigned a bandwidth of $1.02\,MB/s$ to the VS task. We have decided to set the maximum bandwidth at $BW_{tot} = 23,28MB/s$ since, as shown in Fig. 4, the experiments have a success rate of 100%. Each time the number of requests for each channel is higher than the mentioned limits, RPUGuard delays the requests. Hence, when the frequency of the disturbance task is higher than the limit, the transmission latency perceived by the task is very high, but this has no impact on the transmission latency perceived by the VS task that remains completely isolated. This result answers the *[RQ3]* and it demonstrates that it is possible to realize isolated communication channels in software using existing AMP communication mechanisms.

VI. Current Limitations and Future Work

The work carried out during the experiments focuses on the RPUGuard component that has been tested on a Linux kernel without virtualization in order to focus on the isolation capability rather then performances on a specific hypervisor. In addition, the actual implementation does not efficiently utilize all the guaranteed memory bandwidth. Then, a reclaiming mechanism will be added to exploit the unused bandwidth and to improve the best-effort policy. Moreover, this paper focuses on the APU part of the communication assuming that the RPU correctly manages its channel. However, some tests are needed to prove that the separate virtqueue communication channels are effectively isolated, and therefore that APU messages

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.

are not delayed due to high-frequency RPU-driven messages. Finally, since next generation RPUs will feature hardware virtualization support, it could be interesting to realize a virtualization layer on RPU to isolate different provided RPU services to enable interesting scenarios such as the ones discussed in the use case section (UC#2) and relaxing our assumptions A1 and A2.

VII. Conclusions

In this paper, we have analyzed the need and proposed an architectural solution for RPU virtualization. Several use cases in the area of nuclear fusion are discussed to highlight the usefulness of our proposal solution in a real scenario. The implementation focus on the RPUGuard component that aims to abstract the utilization of RPU to multiple VMs, providing an isolated channel with a fixed bandwidth to each VM. Our evaluation with Zynq Ultrascale + (ZCU 104) benchmarks shows the limitation of current AMP communication mechanisms and confirms that RPUGuard is capable of providing isolation under heavy disturbance workload. The proposed work addresses one of many problems to be solved in order to enable safety-critical hard real-time software to be ready to run on a virtualized environment. Hence, future work will focus on other important aspects to provide a safe and isolated environment for critical virtual machines.

References

[1] A. Cilardo, M. Cinque, L. De Simone, and N. Mazzocca, "Virtualization over Multiprocessor System-on-Chip: an Enabling Paradigm for Industrial IoT," *arXiv preprint arXiv:2112.15404*, 2021.

[2] J. Ruh and W. Steiner, "The need for deterministic virtualization in the industrial internet of things," in *Proc. Workshop on Fog Computing and the IoT*, 2019, pp. 26–30.

[3] IEEE, "IEEE 1934-2018 - IEEE standard for adoption of OpenFog reference architecture for Fog Computing," 2018.

[4] A. Burns and R. Davis, "Mixed criticality systems-a review," *Department of Computer Science, University of York, Tech. Rep*, pp. 1–69, 2013.

[5] W. Wolf, A. A. Jerraya, and G. Martin, "Multiprocessor system-on-chip (MPSoC) technology," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 10, pp. 1701–1713, 2008.

[6] A. A. Jerraya and W. Wolf, "The what, why, and how of MPSoCs," in *Multiprocessor Systems-on-Chips*. Elsevier, 2005, pp. 1–18.

[7] H. Youness, M. Moness, and M. Khaled, "MPSoCs and multicore microcontrollers for embedded PID control: A detailed study," *IEEE Trans. on Industrial Inf.*, vol. 10, no. 4, pp. 2122–2134, 2014.

[8] S. Alonso, J. Lázaro, J. Jiménez, U. Bidarte, and L. Muguira, "Evaluating Latency in Multiprocessing Embedded Systems for the Smart Grid," *Energies*, vol. 14, no. 11, p. 3322, 2021.

[9] H. G. Mohammadi *et al.*, "DeepWind: an accurate wind turbine condition monitoring framework via deep learning on embedded platforms," in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, 2020, pp. 1431–1434.

[10] F. Baum and A. Raghuraman, "Making full use of emerging ARM-based heterogeneous multicore SoCs," in *8th European Congress on Embedded Real Time Software and Systems (ERTS 2016)*, 2016.

[11] Xilinx, "Enabling Virtualization with Xen Hypervisor on Zynq UltraScale+ MPSoCs (White Paper WP474)," 2016.

[12] C. H. Kao, "Survey on Evaluation of IoT Services Leveraging Virtualization Technology," in *Proc. 2020 5th International Conference on Cloud Computing and Internet of Things*, 2020, pp. 26–34.

[13] C. Moratelli, S. Johann, M. Neves, and F. Hessel, "Embedded virtualization for the design of secure IoT applications," in *2016 International Symposium on Rapid System Prototyping (RSP)*, 2016, pp. 1–5.

[14] G. Gracioli *et al.*, "Designing mixed criticality applications on modern heterogeneous MPSoC platforms," in *31st Euromicro Conf. Real-Time Systems (ECRTS 2019)*, 2019.

[15] A. Crespo *et al.*, "Hypervisor-based multicore feedback control of mixed-criticality systems," *IEEE Access*, vol. 6, pp. 50 627–50 640, 2018.

[16] M. Hassan, "Heterogeneous MPSoC for mixed criticality systems: Challenges and opportunities," *arXiv preprint arXiv:1706.07429*, 2017.

[17] A. Biondi, M. Marinoni, G. Buttazzo, C. Scordino, and P. Gai, "Challenges in virtualizing safety-critical cyber-physical systems," in *Proceedings of Embedded World Conference 2018*, 2018, pp. 1–5.

[18] M. Cinque, D. Cotroneo, L. De Simone, and S. Rosiello, "Virtualizing mixed-criticality systems: A survey on industrial trends and issues," *Future Generation Computer Systems*, 2021.

[19] ITER. Iter website. [Online]. Available: https://www.iter.org/

[20] M. Cinque, G. De Tommasi, S. Dubbioso, and D. Ottaviano, "Virtualizing Real-Time Processing Units in Multi-Processor Systems-on-Chip," in *2021 IEEE 6th International Forum on Research and Technology for Society and Industry (RTSI)*, 2021, pp. 329–333.

[21] M. Masmano, I. Ripoll, A. Crespo, and J. Metge, "Xtratum: a hypervisor for safety critical embedded systems," in *11th Real-Time Linux Workshop*, 2009, pp. 263–272.

[22] H. Li, X. Xu, J. Ren, and Y. Dong, "ACRN: a big little hypervisor for IoT development," in *Proc. 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, 2019, pp. 31–44.

[23] J. Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto, "Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems," in *Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020)*, 2020.

[24] P. Lucas, K. Chappuis, M. Paolino, N. Dagieu, and D. Raho, "VOSYS-monitor, a Low Latency Monitor Layer for Mixed-Criticality Systems on ARMv8-A," in *29th Euromicro Conf. Real-Time Systems (ECRTS 2017)*, 2017.

[25] R. Ramsauer, J. Kiszka, D. Lohmann, and W. Mauerer, "Look mum, no VM exits!(almost)," *arXiv preprint arXiv:1705.06932*, 2017.

[26] L. Abeni and D. Faggioli, "Using Xen and KVM as real-time hypervisors," *Journal of Systems Architecture*, vol. 106, p. 101709, 2020.

[27] C.-H. Hong, I. Spence, and D. S. Nikolopoulos, "GPU virtualization and scheduling methods: A comprehensive survey," *ACM Computing Surveys (CSUR)*, vol. 50, no. 3, pp. 1–37, 2017.

[28] A. Vaishnav, K. D. Pham, and D. Koch, "A survey on FPGA virtualization," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 131–1317.

[29] C. Bobda *et al.*, "The Future of FPGA Acceleration in Datacenters and the Cloud," *ACM Trans. Reconfigurable Technology and Systems*, vol. 15, no. 3, pp. 1–42, 2022.

[30] Z. Jiang *et al.*, "BlueVisor: Time-Predictable Hardware Hypervisor for Many-core Embedded Systems," *IEEE Trans. Computers*, 2021.

[31] B. Janßen *et al.*, "Towards a type 0 hypervisor for dynamic reconfigurable systems," in *2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, 2017, pp. 1–7.

[32] W. Wang, M. Bolic, and J. Parri, "pvFPGA: paravirtualising an FPGA-based hardware accelerator towards general purpose computing," *International Journal of High Performance Computing and Networking*, vol. 10, no. 3, pp. 179–193, 2017.

[33] OpenAMP. Openamp-rpmsg-virtio-implementation. [Online]. Available: https://github.com/OpenAMP/open-amp/wiki

[34] R. Russell, "Virtio: towards a de-facto standard for virtual I/O devices," *ACM SIGOPS Operating Sys. Rev.*, vol. 42, no. 5, pp. 95–103, 2008.

[35] "European Research Roadmap to the Realisation of Fusion Energy," https://www.euro-fusion.org/fileadmin/user_upload/EUROfusion/Documents/2018_Research_roadmap_long_version_01.pdf, 2018.

[36] J. Wesson and D. Campbell, *Tokamaks*. Oxford University Press, 2011.

[37] M. Cinque, R. Della Corte, A. Eliso, and A. Pecchia, "RT-CASEs: Container-Based Virtualization for Temporally Separated Mixed-Criticality Task Sets," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, vol. 133, 2019, pp. 5:1–5:22.

[38] G. De Tommasi, "Plasma magnetic control in tokamak devices," *J. Fus. Energy*, vol. 38, no. 3-4, pp. 406–436, 2019.

[39] N. Bao *et al.*, "A Real-Time Disruption Prediction Tool for VDE on EAST," *IEEE Trans. Plasma Sci.*, vol. 48, no. 3, pp. 715–720, 2020.

[40] J. Degrave *et al.*, "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, pp. 414–419, 2022.

[41] G. De Tommasi *et al.*, "A RL-based Vertical Stabilization System for the EAST tokamak," in *2022 American Control Conf. (ACC 2022)*, 2022.

[42] R. Albanese *et al.*, "ITER-like vertical stabilization system for the EAST Tokamak," *Nucl. Fus.*, vol. 57, no. 8, p. 086039, 2017.

Authorized licensed use limited to: University of L'Aquila. Downloaded on January 26,2026 at 10:20:47 UTC from IEEE Xplore. Restrictions apply.